

1. RT-1模型基本信息

RT-1模型的输入输出

- 输入
 - a short history of images from a robot's camera. *13张连续摄像头图像*
 - task descriptions expressed in natural language.

输出

- outputs tokenized actions

模型输入、结构和损失函数定义

Image tokenization

- EfficientNet-B3在ImageNet上进行的预训练模型 *300 * 300 图像 EfficientNet*
- 6 images of resolution 300*300 as input and outputs a spatial feature map of shape 9*9*512 *↓ reshape*
- 将输出的特征图: 9*9*512 转换为81个tokens(flatten the output feature map from the EfficientNet into 81 visual tokens), 和自然语言的任务指令进行融合 *81 * 512*
- 每副图像实现了81个tokens到8个tokens的压缩 (token learner) *↓ token learner*
- 6副图像最后形成6*8=48个tokens, 在时间维度上堆叠 (stack) 各个图像经过独立编码后的特征图 *8 * 512*
- 充分利用预训练模型的参数 *↓ 6副图*
- 保留了时间信息 *48 * 512 时间信息*

Natural Language Instructions Embedding

- 将句子或短文本表示为固定维度的向量, 这些向量捕捉了句子中的语义信息
 - The instruction is first embedded via Universal Sentence Encoder
 - Universal Sentence Encoder是基于Transformer架构的
 - 将在Transformer Encoder处理后的所有词元 (token) 的输出向量进行元素级别的求和 (element-wise sum)
 - USE采用全局求和, BERT常用 [CLS] 输出
- 其他可能的方式
 - 如BERT可以对文本序列进行编码, 其中CLS对应的hidden state可以表示整个句子的含义
 - 也可以基于 (Deep Averaging Network, DAN), 如在word2vec对每个token进行embedding然后进行平均连接出线性层输出到设定的维度的向量

USE: 把一句话变为一个固定长度的语义向量。
不论句子多长 → 输出同一长度

非逐词处理, 而是一句话 → 一个任务信号

Action tokenization

机械臂运动的7个变量

- x, y, z (笛卡尔坐标) 这三个变量代表机器人的末端执行器 (如手或工具在机器臂末端的部分) 在三维空间中的位置, 它们定义了末端执行器沿三个轴的确切位置
- Roll, Pitch, Yaw (姿态) 这三个变量描述了机器人末端执行器在空间中的姿态, 它们定义了末端执行器如何绕 x, y, z 轴旋转
- Gripper Opening (夹爪开合) 夹爪开合变量控制机器人夹爪或手的开合。这通常用一个值来表示, 该值决定了夹爪开合的宽度, 范围可以从完全闭合 (抓住物体) 到完全打开 (释放物体)。这个值表示两个夹爪指尖或夹板之间的距离

底盘运动的3个变量

- 运动变量(movement variable): x, y, yaw

模式切换变量

- a discrete variable to switch between three modes

- 每个变量在变量空间范围内量化成256个区间, 相邻两个时间点的动作可以以变量在量化后空间的delta增量来表示, 通过调整这7个动作维度, 机器人可以精确控制其运动与环境的交互, 使其能够执行诸如拾取物体、将物体放置在特定位置以及操作工具或设备等任务。这些维度对于在各种应用中执行复杂的操作至关重要, 包括装配、包装和服务任务

每个动作向量合成256个离散区间 → 进行分类任务。

- 因果自注意力机制的Transformer解码器架构
 - 自回归式生成
 - 输入为图像文本指令的token和输出action的token序列

1x8图像

[I1, I2, ..., I48, L, A1, A2, ..., A11]

- 图像文本指令的token可以基于双向注意力机制计算
- action的token序列基于因果自注意力机制计算
- 设计特殊的注意力掩码，分成四个子区域
 - 左上角区域（视觉/语言 -> 视觉/语言），双向注意力（Bi-directional Attention）
 - 右下角区域（动作 -> 动作），因果注意力（Causal Attention）
 - 右上角区域（视觉/语言 -> 动作），当生成任何一个动作token时，它可以关注所有的视觉和语言输入
 - 左下角区域（动作 -> 视觉/语言），可以关注所有的视觉和语言输入

	视觉	动作
视觉/语言	视觉+语言 理解	I+L -> A
动作	X	逐步生成动作

- 损失函数基于标准的多个类别的交叉熵损失

模型结构及功能分解

- 视觉信息和语言任务信息的融合（FiLM conditioning layer）
 - This embedding is then used as input to identity-initialized FiLM layer
 - FiLM (Feature-wise Linear Modulation) 层是一种神经网络层，用于有条件地调整网络的特征表示。
 - 这种方法特别适合于需要条件输入（如文本指令、语音等）来调整视觉特征的任务，例如视觉问答（Visual Question Answering）、图像风格转换、强化学习等
 - 使用调制的策略（.\robotics_transformer\film_efficientnet\film_conditioning_layer.py）
 - $\text{FiLM}(V) = \gamma * V + \beta$
 - 初始时刻， $\gamma=1$ ， $\beta=0$
 - identity-initialized FiLM also produces better results when training with an EfficientNet initialized from scratch
 - weight设为0，bias的前面一半设为gamma，后面一半设为beta，一开始以视觉信息作为输入，逐步将语言指令信息进行融合，保证模型训练的稳定性（和ControlNet的Zero convolution的概念有相似之处）

FiLM作为调节器只是改变图像tokens里某些区域

本质是用语言重新标定视觉关注与任务相关区域

TokenLearner

- an element-wise attention module that learns to map a large number of tokens into a much smaller number of tokens.
- subsamples the 81 visual tokens that come out of the pre-trained FiLM-EfficientNet layers to just 8 final tokens that are then passed on to our Transformer layers.
- 自适应地选择和重新组合输入特征图的“token”来实现压缩。
- TokenLearner模块做为Transformer模块的输入数据的预处理模块部分，作为整个模型参数的一部分在训练时进行参数更新
 - 注意力权重掩码（“vanilla attention”的一种应用和变体）
 - 参考实现：.\robotics_transformer\tokenizers\token_learner.py